1. OLS Linear Regression — Derivation of Best-Fit Parameters

Hypothesis and Objective

We assume a linear model where the target vector $\boldsymbol{y}$ is approximated by a linear combination of input features contained in the design matrix $\boldsymbol{X}$ and a parameter weight vector $\boldsymbol{\beta}$:

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{\beta}$$

To find the optimal parameters $\hat{\boldsymbol{\beta}}$, we define the optimization objective to minimize the sum of squared errors $S(\boldsymbol{\beta})$:

$$S(\boldsymbol{\beta}) = \sum_i \mathrm{err}_i^2 = \sum_i (y_i - \hat{y}_i)^2$$

Algebraic Matrix Expansion

Expressed cleanly in matrix notation, the error sum is:

$$S(\boldsymbol{\beta}) = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})$$

Expanding this product via matrix transposition rules, noting that $(\boldsymbol{X}\boldsymbol{\beta})^T = \boldsymbol{\beta}^T\boldsymbol{X}^T$, gives:

$$S(\boldsymbol{\beta}) = \boldsymbol{y}^T\boldsymbol{y} - \boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\beta} - \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

Because the term $\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\beta}$ evaluates to a scalar (1×1 matrix), it is structurally identical to its own transpose:

$$(\boldsymbol{y}^T\boldsymbol{X}\boldsymbol{\beta})^T = \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y}$$

We combine these identical scalar terms to simplify our objective function:

$$S(\boldsymbol{\beta}) = \boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

Differentiation with Respect to $\boldsymbol{\beta}$

To find the parameter vector that minimizes this error, we take the partial derivative of $S(\boldsymbol{\beta})$ with respect to the vector $\boldsymbol{\beta}$ and equate it to zero.

> **Important Exam Note:** As emphasized in class, remember to point out to the examiner that the derivative is taken strictly with respect to the parameters ($\boldsymbol{\beta}$), not the spatial data matrix ($\boldsymbol{X}$).

Using standard matrix calculus identities ($\frac{\partial \boldsymbol{\beta}^T\boldsymbol{A}}{\partial \boldsymbol{\beta}} = \boldsymbol{A}$ and $\frac{\partial \boldsymbol{\beta}^T\boldsymbol{A}\boldsymbol{\beta}}{\partial \boldsymbol{\beta}} = 2\boldsymbol{A}\boldsymbol{\beta}$), we evaluate the gradient:

$$\frac{\partial S(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = \frac{\partial}{\partial \boldsymbol{\beta}}\left(\boldsymbol{y}^T\boldsymbol{y} - 2\boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{y} + \boldsymbol{\beta}^T\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}\right)$$

$$0 = -2\boldsymbol{X}^T\boldsymbol{y} + 2\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta}$$

Isolating the Best-Fit Weight Formula

Rearranging terms yields the **Normal Equations**:

$$\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{X}^T\boldsymbol{y}$$

Multiplying both sides from the left by the inverse matrix $(\boldsymbol{X}^T\boldsymbol{X})^{-1}$ isolates our final parameter weight formula:

$$\hat{\boldsymbol{\beta}} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$$

2. Nonlinear Least Squares — Derivation of the Iterative Step ($\boldsymbol{\delta}$)

Hypothesis and Linearization

When our model function $f(x_i, \boldsymbol{\beta})$ is nonlinear with respect to its parameters $\boldsymbol{\beta}$, it cannot be solved analytically via basic matrix inversion. Instead, we linearize the function in the immediate neighborhood of a current parameter guess ($\bar{\boldsymbol{\beta}}$) using a first-order Taylor series expansion:

$$f(x_i, \bar{\boldsymbol{\beta}} + \boldsymbol{\delta}) \approx f(x_i, \bar{\boldsymbol{\beta}}) + \boldsymbol{J}_i\boldsymbol{\delta}$$

Where $\boldsymbol{\delta}$ is the correction step vector we want to solve for , and $\boldsymbol{J}_i$ represents rows of the **Jacobian Matrix** containing the partial derivatives evaluated at our current guess:

$$\boldsymbol{J}_i = \frac{\partial f(x_i, \boldsymbol{\beta})}{\partial \boldsymbol{\beta}}$$

Setting up the Linearized Cost Surface

Our goal shifts to minimizing the localized sum of squares with respect to our step vector $\boldsymbol{\delta}$:

$$S(\bar{\boldsymbol{\beta}} + \boldsymbol{\delta}) \approx \sum_i \left(y_i - [f(x_i, \bar{\boldsymbol{\beta}}) + \boldsymbol{J}_i\boldsymbol{\delta}]\right)^2$$

Rewriting this cleanly in vector form:

$$S(\bar{\boldsymbol{\beta}} + \boldsymbol{\delta}) \approx \left(\boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}}) - \boldsymbol{J}\boldsymbol{\delta}\right)^T \left(\boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}}) - \boldsymbol{J}\boldsymbol{\delta}\right)$$

Let $\boldsymbol{r} = \boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}})$ represent the vector of current residuals. Substituting this back into the matrix formulation yields:

$$S(\bar{\boldsymbol{\beta}} + \boldsymbol{\delta}) \approx (\boldsymbol{r} - \boldsymbol{J}\boldsymbol{\delta})^T(\boldsymbol{r} - \boldsymbol{J}\boldsymbol{\delta}) = \boldsymbol{r}^T\boldsymbol{r} - 2\boldsymbol{\delta}^T\boldsymbol{J}^T\boldsymbol{r} + \boldsymbol{\delta}^T\boldsymbol{J}^T\boldsymbol{J}\boldsymbol{\delta}$$

Optimization Differentiation

To minimize the localized cost surface, we take the derivative with respect to the step vector $\boldsymbol{\delta}$ and equate it to zero:

$$0 = \frac{\partial S(\bar{\boldsymbol{\beta}} + \boldsymbol{\delta})}{\partial \boldsymbol{\delta}} = -2\boldsymbol{J}^T \boldsymbol{r} + 2\boldsymbol{J}^T \boldsymbol{J}\boldsymbol{\delta}$$

Substituting our residual vector $\boldsymbol{r} = \boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}})$ back into the derivative equation:

$$0 = -2\boldsymbol{J}^T [\boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}})] + 2\boldsymbol{J}^T \boldsymbol{J}\boldsymbol{\delta}$$

Final Step Matrix Equation

Isolating the parameters yields the system of linear equations used to compute the correction step at each iteration:

$$\left(\boldsymbol{J}^T \boldsymbol{J}\right)\boldsymbol{\delta} = \boldsymbol{J}^T \left[\boldsymbol{y} - \boldsymbol{f}(\bar{\boldsymbol{\beta}})\right]$$

3. Logistic Regression — Derivation of the Cross-Entropy Loss Function

Probability Hypothesis

Logistic Regression models classification probabilities using the sigmoid function:

$$P(y_i = 1 \mid x_i, \boldsymbol{w}) = \sigma(\boldsymbol{w}^T \boldsymbol{x}_i + b) = \frac{1}{1 + e^{-(\boldsymbol{w}^T \boldsymbol{x}_i + b)}}$$

Because this is a binary system, the probability of encountering the negative class ($y_i = 0$) is:

$$P(y_i = 0 \mid x_i, \boldsymbol{w}) = 1 - P(y_i = 1 \mid x_i, \boldsymbol{w})$$

We can combine these two cases into a single probability density expression for any individual observation i using Bernoulli distribution notation:

$$P(y_i \mid x_i, \boldsymbol{w}) = P_i^{y_i}(1 - P_i)^{1-y_i}$$

Likelihood Function

Assuming that all data observations are independent and identically distributed (i.i.d.), the total joint probability (the Likelihood, L) across the entire dataset is the product of their individual probabilities:

$$L(\boldsymbol{w}) = \prod_i P(y_i \mid x_i, \boldsymbol{w}) = \prod_i P_i^{y_i}(1 - P_i)^{1-y_i}$$

Log-Likelihood Transformation

To convert these products into sums, we take the natural logarithm of both sides:

$$\ln L(\boldsymbol{w}) = \ln\left(\prod_i P_i^{y_i}(1-P_i)^{1-y_i}\right)$$

$$\ln L(\boldsymbol{w}) = \sum_i [y_i \ln(P_i) + (1-y_i)\ln(1-P_i)]$$

Formulating the Loss Minimization Objective

Machine learning optimizers are designed to minimize cost functions rather than maximize them. To turn maximizing the likelihood into a minimization problem, we multiply the expression by -1. This directly gives the **Binary Cross-Entropy Loss** formula required by your syllabus:

$$l(\boldsymbol{w}) = -\sum_i \ln P(y_i \mid x_i, \boldsymbol{w})$$

$$l(\boldsymbol{w}) = -\sum_i [y_i \log(P_i) + (1-y_i)\log(1-P_i)]$$

4. Support Vector Machines — Soft-Margin Optimization Objective

Structural Hypothesis & Slack Variables

SVMs maximize the separation margin between classes. To handle non-linearly separable data or noisy outliers, we use a soft-margin formulation that introduces non-negative slack variables (ξ_i).

The classification constraint for each point becomes:

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \xi_i \quad \forall \boldsymbol{x}_i$$

$$\text{with } \xi_i \geq 0$$

Primal Objective Formulation

The regularization parameter C balances maximizing the margin width (by minimizing $\frac{1}{2}||\boldsymbol{w}||^2$) against minimizing the total distance of margin violations:

$$\min_{\boldsymbol{w},b} \frac{||\boldsymbol{w}||^2}{2} + C\sum_i \xi_i$$

Lagrangian Formulation

To solve this constrained optimization problem, we introduce Lagrange multipliers ($\alpha_i \geq 0$ and $\mu_i \geq 0$) to construct the unconstrained Lagrangian function:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2}||\boldsymbol{w}||^2 + C\sum_i \xi_i - \sum_i \alpha_i \left[y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1 + \xi_i\right] - \sum_i \mu_i\xi_i$$

To find the minimum, we take the partial derivatives of $\mathcal{L}$ with respect to the primal variables ($\boldsymbol{w}, b, \boldsymbol{\xi}$) and set them to zero:

1. $$\frac{\partial \mathcal{L}}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_i \alpha_i y_i \boldsymbol{x}_i = 0 \implies \boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$$
2. $$\frac{\partial \mathcal{L}}{\partial b} = -\sum_i \alpha_i y_i = 0 \implies \sum_i \alpha_i y_i = 0$$
3. $$\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \implies \alpha_i + \mu_i = C$$

Substituting back into the Dual Function

Substituting these primal expressions back into the original Lagrangian eliminates $\boldsymbol{w}, b$, and $\boldsymbol{\xi}$, yielding the standard **Wolfe Dual Optimization** problem:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j (\boldsymbol{x}_i^T\boldsymbol{x}_j)$$

$$\text{subject to } 0 \leq \alpha_i \leq C \quad \text{and} \quad \sum_i \alpha_i y_i = 0$$

5. Regularized Regression Cost Functions (Ridge and Lasso)

When mitigating overfitting issues or addressing feature correlation, regularized regression techniques modify the standard Ordinary Least Squares (OLS) optimization surface by appending structural coefficient parameter penalties.

Ridge Regression Cost Function (L_2 Regularization)

Ridge regression shrinks parameter coefficients smoothly by penalizing the square of their magnitudes. This controls high variance without forcing weights to zero completely:

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{m}\left(y_i - \boldsymbol{x}_i^T\boldsymbol{\beta}\right)^2 + \alpha\sum_{j=1}^{n}\beta_j^2$$

Lasso Regression Cost Function (L_1 Regularization)

Lasso regression applies an absolute value penalty to the weights. This geometric property drives uninformative feature weights to exactly zero, turning Lasso into an automated feature selection framework:

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{m} \left(y_i - \boldsymbol{x}_i^T \boldsymbol{\beta}\right)^2 + \alpha \sum_{j=1}^{n} |\beta_j|$$

Where:

- $\sum_{i=1}^{m} \left(y_i - \boldsymbol{x}_i^T \boldsymbol{\beta}\right)^2$ is the baseline Residual Sum of Squares (RSS).
- α (or λ in some texts) is the tuning hyperparameter that sets the intensity of the regularization penalty. When $\alpha = 0$, both formulas revert back to standard OLS linear regression parameters.